

SZYMON ŻYREK

SYSTEMS & PRODUCT ENGINEER · HANDS-ON SOLUTION ARCHITECT · AI-NATIVE SOFTWARE DELIVERY

stynkdev@gmail.com | +48 505 784 179 | Tricity, Poland | Remote | Available immediately | [LinkedIn](#) | [GitHub](#)

ARCHITECTURE LEARNED FROM THE IMPLEMENTATION SIDE

12+ years implementing the transitions architecture is meant to enable - from debugging inherited code, through integration and selective extraction, to distributed systems, platforms, production ownership, UX and agentic delivery. I now use that accumulated perspective to translate business problems into complete solutions whose technical complexity disappears behind the right abstraction.

BECOMING BY DOING

MISYS → FINASTRA 2013-2021 · Gdynia

JUNIOR → EXPERT

- STAGE 1 · Junior → independent engineer



DEBUGGING LEGACY RELIABILITY

PROBLEM

Inherited financial-risk systems arrived with outdated setup notes, unfamiliar repositories, mixed C++/Java EE stacks and no original teams to explain them.

INTERVENTION

Built a repeatable investigation workflow around logs, debuggers, stack traces, Git history/bisect, Bash, tests, documentation, source code and finding the right human when needed.

EVIDENCE

Regularly reproduced and fixed failures in systems I had not known existed before receiving the task.

COMPETENCE

Learned that engineering independence is not knowing every technology; it is knowing how to make an unknown system reveal the truth.
- STAGE 2 · Engineer - integration & modernization



INTEGRATION DISTRIBUTED CI/CD IDENTITY

PROBLEM

Stabilized legacy products now had to participate in common CI, messaging, SSO and cross-product workflows without rewriting everything that already worked.

INTERVENTION

Worked with the architect as the legacy-side expert: added adapters and common contracts, JMS/event flows, OpenID integration and deployment standardization; treated mature products as black boxes with useful capabilities.

EVIDENCE

Helped move multiple heterogeneous products toward common platform contracts while preserving their proven internals.

COMPETENCE

Learned to separate implementation from capability: stable seams, protocols and contracts let very different systems become composable.
- STAGE 3 · Senior - selective extraction & subsystem ownership



SCALING DISTRIBUTED IDENTITY PLATFORM

PROBLEM

Some useful legacy capabilities could not scale or integrate cleanly because the bottleneck lived in centralized synchronization, data access or incompatible identity models.

INTERVENTION

Extracted scalable/stateless processing around existing calculation engines and built User Management to reconcile one platform identity with system-specific users, permissions and synchronized data.

EVIDENCE

Owned User Management as other teams integrated with it; became a cross-team contact and mentor while the extracted processing path enabled horizontal scale without rewriting the whole engine.

COMPETENCE

Learned architecture as constraint reconciliation: change the boundary that blocks the required property, preserve what already works, and hide necessary local differences under one coherent abstraction.
- STAGE 4 · Expert - platform perspective



PLATFORM INTEGRATION DISTRIBUTED BUSINESS

PROBLEM

Years of acquisitions had left many products, eras and stacks; the business wanted one coherent product instead of separately exposing and supporting each inherited system.

INTERVENTION

Participated in the final composition: common identity, messaging, contracts and deployment exposed capabilities through a unified platform where clients could define calculation workflows and dashboards above the legacy engines.

EVIDENCE

Saw heterogeneous products become a configurable financial platform rather than a collection of applications.

COMPETENCE

Internalized the idea that a system is an abstraction over capabilities - and a platform is a language for composing them into new solutions.
- Intel Corporation - Firmware Engineer

Oct 2021 - Mar 2022 · Gdańsk



FIRMWARE DEBUGGING DATA

PROBLEM

SSD firmware exposed timing, state, memory and hardware constraints normally hidden by managed enterprise runtimes.

INTERVENTION

Worked on NAND SSD firmware fixes in Embedded C and Python test automation, investigating failures across the software/hardware boundary.

EVIDENCE

Delivered firmware and test work inside a specialized SSD development environment.

COMPETENCE

Learned firsthand what higher-level abstractions buy - and which underlying hardware/runtime constraints they simplify rather than erase.

BECOMING BY DOING - CONTINUED

- Ciklum / EverC - Full Stack Developer

Nov 2022 - Aug 2023 · Remote

OPERATIONS OBSERVABILITY RELIABILITY LEGACY

PROBLEM

A fast-moving fraud-detection product had to keep shipping while the team also owned deployment and production reliability end to end.

INTERVENTION

Worked mainly on the Grails/Groovy backend, deployments and on-call; restored failed services under time pressure and investigated root causes through monitoring and logs.

EVIDENCE

Owned software behavior beyond merge: delivery, observability, incident response and follow-up investigation were one engineering loop.

COMPETENCE

Learned that production ownership makes maintainability and observability engineering requirements; velocity lasts only while a system remains understandable and safe to change.
-
- Nordea - Full-stack Developer

Aug 2023 - Mar 2024 · Gdynia

GOVERNANCE SECURITY INTEGRATION CI/CD
- PROBLEM

New credit-product features entered a banking ecosystem too large for any one team to control, so work began with discovering data owners, contracts, access paths and non-functional requirements.
- INTERVENTION

Built Java/Angular microservice and microfrontend flows in team-owned components: teams chose internals, while precise external contracts and strict security, performance, testing and integration gates protected the ecosystem.
- EVIDENCE

Delivered product flows where independently owned components could evolve internally yet remain composable through stable contracts and standardized validation.
- COMPETENCE

Learned that architecture at organizational scale moves coherence from shared implementation rules to ownership boundaries, stable contracts and independently verifiable behavior - while governance itself must scale and automate.

Hapag-Lloyd AG - Full Stack Engineer

Jun 2024 - Mar 2025 · Gdańsk

INTEGRATION DATA PLATFORM UX

PROBLEM

Global logistics reconciled events and state across the transport chain inside a mature, strongly opinionated internal DDD platform.

INTERVENTION

Worked on real-time container-tracking/event flows and maintenance through higher-level framework abstractions for domain models, validation and UI rather than rebuilding plumbing per feature.

EVIDENCE

Contributed inside a mature platform that hid much of the technical machinery behind domain-oriented building blocks.

COMPETENCE

Experienced platform abstractions from the consumer side; it reinforced that the key design problem is the boundary between the user/domain model and the machinery that makes the abstraction true.

Independent R&D - AI-assisted software delivery

Apr 2025 - Sep 2025 · Poland

AI/AGENTS AUTOMATION PLATFORM CI/CD

PROBLEM

Architectural ideas were cheap to conceive but expensive to validate deeply because implementation latency limited experimentation.

INTERVENTION

Used coding agents as an execution multiplier and experimented with documentation-driven context, reviewer agents, evidence gates, triggerable roles and local-model workflows.

EVIDENCE

Turned architectural ideas into working artifacts fast enough to iterate on the delivery process itself, not only on the code.

COMPETENCE

Learned to architect software delivery itself: context, responsibility boundaries, feedback loops, validation and escalation determine whether agent autonomy stays reliable.

Stynk - Independent Systems & Product Engineer / Technical Owner

Oct 2025 - Present · Poland

REQUIREMENTS BUSINESS UX PLATFORM OPERATIONS SECURITY AI/AGENTS

PROBLEM

A growing construction company could not scale because the owner was the human message bus: information lived in calls, paper, private notes, email and people's memory, with no reliable shared state.

INTERVENTION

Did end-to-end discovery, modeled domain/workflows, designed UX and security, built and operate the CRM/site/infrastructure, moved coordination into auditable notifications/escalations, and generalized changing business logic into configurable Studio models and rules.

EVIDENCE

~20-25 users handle ~120-150 contracts/year in one searchable system; since launch there has been no data-loss incident or user-blocking outage. Delivery runs through staging/CI/E2E/preflight/rollback automation and a multi-agent loop that mostly escalates only real product decisions.

COMPETENCE

Integrated the earlier lessons into one practice: discover the bottleneck, choose the right abstractions, expose them in the user's language, automate guarantees underneath, and keep changing business models owned by the business.

WHAT THAT ADDS UP TO

I now work best where business space meets implementation space. I can discover an ambiguous operational problem, model the domain and ownership boundaries, design the architecture and UX that expose the right capability, and own the result through production. I did not learn that as a catalogue of architecture patterns; I learned it by implementing the transitions - and Stynk is the first place I have consciously applied the whole chain end to end.